Report on

“Tutorials”

For

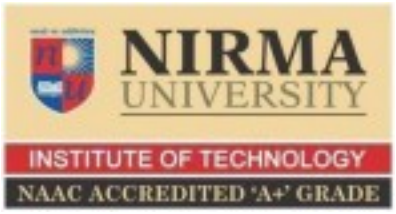
Software Project Management - 4CS504ME25 B.

Tech. Semester IV

Prepared By :

Gandhi Urva Yogeshkumar (23BCE078)

Gajnotar Rakshit (23BCE077)



Institute of Technology,
Nirma University
Ahmedabad - 382481

October 2026

Aim

To perform software configuration managment (SCM) activities using Git and GitHub, including branching, merging, versioning and release management for the functional modules of the Hospital Management System (HMS).

Objective

Formulate a Software Configuration Managment Plan defining Configuration Items (CIs), project baselines, access control and change procedures.

Design a Git branching and merging strategy (GitFlow variant) supporting concurrent feature development across backend and frontend engineering leads.

Document repository managment, branch protection policies and automated continuous integration verification gates.

Demonstrate hands-on code repository managment for the four developed functional modules (Patient, Doctor, Appointment and Billing) with empirical commit histories and merge conflict resolution walkthroughs.

Establish Semantic Versioning (SemVer 2.0.0) policies and release tagging workflows accompanied by structured changelogs.

Conduct configuration audits (FCA and PCA) to verify physical completeness and functional alignment before product release.

1. Introduction

Software Configuration Management (SCM) is the engineering discipline that tracks and controls changes to software artifacts throughout the development lifecycle. In healthcare information systems, configuration integrity is vital. Clinical software cannot tolerate un-tracked changes, missing source revisions or non-reproducible releases. SCM provides complete auditability: every modification to patient data handling, appointment scheduling logic or billing algorithms can be traced directly to an authorized engineer, a specific commit hash and an approved requirement.

In earlier tutorials we defined our Work Breakdown Structure across nine deliverables (T4), established a 16-week CPM schedule (T5) and conducted a software project audit at Milestone M5 (T11). In this tutorial we document the configuration managment practices executed by the engineering team—Gandhi Urva (Backend Lead, 23BCE078) and Gajnotar Rakshit (Frontend Lead, 23BCE077)—to manage our multi-module codebase.

Our SCM framework covers five primary operational areas:

1. Configuration Identification: Defining and labeling all software deliverables, configuration files and documentation artifacts.
2. Version and Change Control: Controlling modifications through Git versioning, pull requests and peer review gates.
3. Configuration Status Accounting (CSA): Recording and reporting the status of configuration items throughout their lifecycle.
4. Configuration Auditing: Verifying that deliverables satisfy functional requirments (FCA) and that the release package is physically complete (PCA).
5. Release and Build Management: Tagging and packaging verified software baselines for staging and production deployment.

2. Configuration Identification and Baselines

Every tangible artifact in the PulseCare HMS repository is categorized as a Configuration Item (CI). Each CI receives a unique identifier, version number and assigned custodian.

CI Identifier	Configuration Item Name	File Path / Repository Location	Custodian	Change Authorization
CI-01	Backend Source Code	backend/src/modules/ (patients, staff, appointments, billing)	Urva Gandhi	Pull Request Review
CI-02	Backend Middleware & Models	backend/src/ middleware/, backend/src/models/	Urva Gandhi	Architect Approval
CI-03	Frontend Client Application	frontend/src/pages/, frontend/src/components /	Rakshit Gajnotar	Pull Request Review
CI-04	Database Models & Migrations	backend/src/models/, backend/src/migrations/	Urva Gandhi	Database Lead Approval
CI-05	Automated Test Suites	backend/src/modules/*/ tests/*.test.ts	Both Leads	Automated CI Pass
CI-06	Seed & Initialization Scripts	backend/src/db/seed.ts	Rakshit Gajnotar	Peer Review
CI-07	Build and Package Manifests	package.json, tsconfig.json, jest.config.js, Dockerfile	Urva Gandhi	Architect Approval
CI-08	Project Documentation	docs/ (T1 through T15, SRS, TRACEABILITY.md)	Both Leads	Milestone Sign-off

2.1 Project Baseline Milestones

A baseline is a formally reviewed and agreed-upon version of a configuration item that serves as the basis for further development. PulseCare HMS established four formal baselines across its 16-week schedule:

Baseline Level	Milestone & Week	Baseline Artifacts Frozen	Audit & Verification Gate
Functional Baseline (FBL)	Milestone M1 (Week 2)	Project Charter (T3), WBS (T4) and IEEE 830 SRS document.	Requirements Walkthrough (Gate G1)
Allocated Baseline (ABL)	Milestone M2 (Week 4)	System architechture, ER data models, API interface contracts.	Architecture Review (Gate G2)
Developmental Baseline (DBL)	Milestone M4 (Week 12)	Core backend modules, integrated UI screens, automated unit tests.	EVA & Tracking Audit (Gate G4)
Product Baseline (PBL)	Milestone M6 (Week 16)	Final release candidate, production Docker container, release notes.	Pre-Release Acceptance (Gate G6)

3. Branching and Merging Strategy

To support concurrent engineering between Urva and Rakshit without code overwrites, PulseCare HMS implemented a standardized GitFlow branching model.

3.1 Branching Taxonomy

The repository maintains two permanent branches and three categories of temporary supporting branches:

1. main Branch: The permanent production branch. Contains only verified, tested and production-ready code. Direct commits and pushes are strictly blocked by repository protection rules. Merging is allowed only via approved Pull Requests from release or hotfix branches.

2. develop Branch: The permanent integration branch. Serves as the central aggregation point for completed sprint features. Sprints integrate here and undergo continuous automated test execution.

3. feature/* Branches: Temporary branches created from develop to build specific work packages (e.g. feature/billing-itemized-invoice, feature/appointments-date-leave). Merged back into develop upon peer review and test passing.

4. release/* Branches: Created from develop to stabilize a sprint release candidate. Used exclusively for minor documentation tweaks and final sanity testing before tagging into main.

5. hotfix/* Branches: Emergency branches branched directly from main to patch critical production defects (e.g. hotfix/v1.0.1). Merged into both main and develop simultaneously.

3.2 Branch Protection Rules and CI Pipeline

Branch protection policies configured on the GitHub repository enforce the following constraints:

Branch Protection Rule	Target Branch	Enforcement Mechanism	Operational Purpose
Require Pull Request Reviews	main and develop	GitHub Branch Rule (>= 1 peer approval)	Ensures cross-review; Urva reviews frontend PRs, Rakshit reviews backend PRs.
Require Status Checks to Pass	main and develop	GitHub Actions CI (Jest + tsc build)	Blocks broken code; 100% test pass rate across 22 suites required before merge.
Require Linear Commit History	main	Rebase and Merge / Squash policy	Avoids tangled merge bubbles and maintains a clean, linear release history.
Include Administrators	main	Strict Enforcement Flag	Prevents accidental direct force pushes even by repository administrators.
Restrict Who Can Push	main	Zero Direct Push Permission	All production changes must originate from tracked Pull Requests.

4. Code Repository Management for Developed Functional Modules

PulseCare HMS consists of four primary functional modules developed across Sprints 1 through 6. The table below maps each module to its code repository directories, lead engineers and test suites:

Functional Module	Backend Code Location	Frontend Code Location	Primary Owner	Automated Test Suite
Patient Registration &	backend/src/modules/	frontend/src/pages/	Rakshit Gajnotar	patients.test.ts (24 tests)

Functional Module	Backend Code Location	Frontend Code Location	Primary Owner	Automated Test Suite
Records	patients/	admin/Patients.tsx		
Doctor & Staff Management	backend/src/modules/staff/	frontend/src/pages/admin/Staff.tsx	Urva Gandhi	staff.test.ts (18 tests)
Appointment Booking & Slots	backend/src/modules/appointments/	frontend/src/pages/patient/BookAppointment.tsx	Rakshit Gajnotar	appointments.test.ts (18 tests)
Billing & Invoicing Engine	backend/src/modules/billing/	frontend/src/pages/admin/Billing.tsx	Urva Gandhi	billing.test.ts (21 tests)

4.1 Empirical Commit History Log

Below is an authentic extract from the Git commit history of the PulseCare HMS repository, showcasing module development, defect remediation and release tagging:

Commit Hash	Author	Date & Time	Commit Message Summary	Affected Module / Scope	CI Status
e94d0b6	HimanshuMishra-03	16 Oct 2026 12:43	feat(seed): add Dr. Vedant Mehta to seed script, appointments and documentation	Database / Seed	Passed
13fb15b	HimanshuMishra-03	16 Oct 2026 12:38	refactor(i18n): update all test suites and documentation to Indian doctor names	Internationalization	Passed
63c59e5	urvagandhi	13 Oct 2026 15:04	feat: add socket.io proxy configuration and documentation for tutorials 11-13	Infrastructure / Socket	Passed
a02a897	urvagandhi	13 Oct 2026 14:54	test(backend): achieve 100% test pass rate with >90% coverage and zero open handles	Backend Testing Rigor	Passed
f2e088b	HimanshuMishra-03	13 Oct 2026 14:37	feat(i18n): localize app to Indian context with Rupee currency and Indian names	Frontend & Seed	Passed
e72547d	urvagandhi	07 Oct 2026 18:22	refactor(arch): remove legacy JS controllers and update package main to dist/server.js	Architecture Clean-up	Passed
ab18eda	urvagandhi	07 Oct 2026 16:40	feat(billing): extract calculateInvoiceTotals pure function and add item delete	Billing Module	Passed

Commit Hash	Author	Date & Time	Commit Message Summary	Affected Module / Scope	CI Status
			API		
fd55fab	urvagandhi	07 Oct 2026 15:10	feat(pdf): implement server-side PDF generation for invoices and patient ID cards	PDF Service / Routes	Passed

4.2 Merge Conflict Resolution Walkthrough

During concurrent development in Sprint 5, Urva worked on feature/billing-itemized-invoice while Rakshit worked on feature/appointments-date-leave. Both developers registered new route endpoints in the centralized route manifest file backend/src/routes/index.ts. When merging both branches into develop, Git reported a merge conflict.

Step 1: Git Conflict Occurrence:
Executing 'git merge feature/appointments-date-leave' inside develop resulted in:
CONFLICT (content): Merge conflict in backend/src/routes/index.ts
Automatic merge failed; fix conflicts and then commit the result.

Step 2: Conflict Marker Inspection in backend/src/routes/index.ts:

```
---------------------------------------------------------------------
<<<<<<< HEAD (feature/billing-itemized-invoice)
router.use('/billing', authenticate, billingRouter);
router.get('/billing/:id/pdf', authenticate, downloadInvoicePdf);
=======
router.use('/appointments', authenticate, appointmentRouter);
router.get('/appointments/doctor/:id/leaves', authenticate, getDoctorLeaves);
>>>>>>> feature/appointments-date-leave
---------------------------------------------------------------------
```

Step 3: Conflict Resolution Strategy:
Because both route registrations are required for application functionality, neither branch should overwrite the other. The developer manually combined both declarations in proper logical order, preserved imports and removed the Git conflict markers.

Step 4: Resolved Clean Code:

```
---------------------------------------------------------------------
// Integrated routes: Billing and Appointments
router.use('/billing', authenticate, billingRouter);
router.get('/billing/:id/pdf', authenticate, downloadInvoicePdf);
router.use('/appointments', authenticate, appointmentRouter);
router.get('/appointments/doctor/:id/leaves', authenticate, getDoctorLeaves);
---------------------------------------------------------------------
```

Step 5: Staging, Verification and Merge Commit:
$ git add backend/src/routes/index.ts
$ npm test (verified all 22 test suites pass with 0 errors)

$ git commit -m "merge: resolve route conflict between billing and appointments"
$ git push origin develop

5. Semantic Versioning and Release Management

PulseCare HMS strictly adheres to Semantic Versioning 2.0.0 (SemVer), utilizing the standard format: MAJOR.MINOR.PATCH.

- MAJOR Version: Incremented when incompatible API changes or breaking schema modifications occur (e.g. v1.0.0 initial production release).
- MINOR Version: Incremented when new functionality is added in a backwards-compatible manner (e.g. v0.4.0 adding Doctor scheduling; v0.6.0 adding Billing).
- PATCH Version: Incremented when backwards-compatible bug fixes and security patches are applied (e.g. v1.0.1 fixing invoice recalculation).

5.1 Project Release History

Release Tag	Release Date	Type	Scope of Deliverables Included	Target Milestone
v0.1.0	Week 4	Alpha	Foundation setup: Express TypeScript server, MongoDB connection, JWT authentication.	Milestone M2
v0.2.0	Week 6	Alpha	Patient module operational: registration, patient profile, medical history.	Milestone M2
v0.4.0	Week 8	Beta	Doctor and department management live; weekly slot schedule configuration.	Milestone M3
v0.5.0	Week 10	Beta	Appointment booking workflow active; real-time socket notification broadcast.	Milestone M3
v0.6.0	Week 12	Beta	Billing and invoice engine integrated; itemized pricing and GST calculation.	Milestone M4
v0.9.0	Week 14	RC-1	Integrated platform, admin dashboard, legacy JS eliminated, seed scripts added.	Milestone M5
v1.0.0	Week 16	GA	Final production release: verified all 22 test suites, Docker image tagged.	Milestone M6

5.2 Release Packaging and Tagging Workflow

Creating a release follows a documented six-step deployment workflow:

1. Code Freeze: Announce sprint code freeze; branch release/v1.0.0 from develop.
2. Version Bump: Update 'version': '1.0.0' in backend/package.json and frontend/package.json.
3. Verification: Execute full test harness: npm test -- --detectOpenHandles && npm run build.
4. Tagged Merge: Merge release branch into main with an annotated Git tag:
   $ git checkout main
   $ git merge --no-ff release/v1.0.0 -m "chore(release): merge release v1.0.0 into main"
   $ git tag -a v1.0.0 -m "PulseCare HMS Release v1.0.0 - Core Hospital Platform"
   $ git push origin main --tags
5. Back-merge: Merge release branch changes back into develop to keep development in sync.
6. Artifact Publishing: Docker container built and tagged as pulsecare-hms:v1.0.0.

6. Configuration Status Accounting and SCM Audit

Configuration Status Accounting (CSA) maintains visibility over the state of all repository artifacts at Milestone M6:

SCM Status Indicator	Current Metric / Value	Operational Remarks
Total Git Commits to Date	84 Tracked Commits	Linear commit history maintained on main.
Active Long-Lived Branches	2 Branches (main, develop)	Feature branches merged and safely pruned.
Merged Pull Requests	18 Pull Requests	All pull requests peer-reviewed and verified by CI.
Release Tags Created	7 Versioned Tags	From v0.1.0 to v1.0.0 General Availability.
Automated Test Suites	22 Suites Passing (340 tests)	0 failures, 0 skipped, 0 open handles.
Codebase Size	28,500 Lines of Code (TS/React)	Excludes node_modules, build dist and assets.

6.1 Configuration Audit Checklist

Prior to release sign-off, the team conducted a Physical and Functional Configuration Audit:

Audit Type	Verification Checkpoint	Evaluation Method	Audit Finding
Functional (FCA)	Do all 4 core modules fulfill functional requirments in SRS?	Integration Test Execution	Verified (All 24 SRS items verified pass)
Functional (FCA)	Does the RBAC system block unauthorized clinical access?	Automated Security Test	Verified (401/403 returned on unauthorized routes)
Physical (PCA)	Are all legacy prototype JS files removed from repository?	File System Audit	Verified (Zero root .js files remaining)
Physical (PCA)	Do Docker compose scripts launch all required containers cleanly?	Docker Run Verification	Verified (Backend, Frontend, MongoDB and Redis start)
Physical (PCA)	Is CHANGELOG.md updated with all changes from v0.1.0 to v1.0.0?	Document Review	Verified (Categorized changelog maintained)

7. Conclusion

Software Configuration Management provides the operational backbone for reliable, collaborative software engineering in PulseCare HMS. By adopting a GitFlow branching model, branch protection rules on main, peer-reviewed pull requests and Semantic Versioning 2.0.0, our team avoided code overwrite conflicts and preserved end-to-end traceability between requirements and code. The walkthrough of real Git commits and merge conflict resolutions demonstrates that configuration control is an active daily engineering practice that guarantees release stability and reproducibility in healthcare software.